Employee_Turnover_Project

In [1]:
```
import pandas as pd
df = pd.read_csv("employee_turnover.csv")
df.head()
```

Out[1]:

	Job_Satisfaction	Performance_Rating	Years_At_Company	Work_Life_Balance	Distanc
0	0.562326	0.141129	0.123989	0.347583	
1	0.017041	0.559047	0.511203	0.793908	
2	0.774699	0.604371	0.798174	0.260500	
3	0.628174	0.385249	0.230104	0.516809	
4	0.799183	0.199967	0.839029	0.247927	

In [3]:
```
df.shape
```

Out[3]:
```
(1350, 16)
```

In [5]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1350 entries, 0 to 1349
Data columns (total 16 columns):
 #   Column                                  Non-Null Count  Dtype
---  ------                                  --------------  -----
 0   Job_Satisfaction                        1350 non-null   float64
 1   Performance_Rating                      1350 non-null   float64
 2   Years_At_Company                        1350 non-null   float64
 3   Work_Life_Balance                       1350 non-null   float64
 4   Distance_From_Home                      1350 non-null   float64
 5   Monthly_Income                          1350 non-null   float64
 6   Education_Level                         1350 non-null   float64
 7   Age                                     1350 non-null   float64
 8   Num_Companies_Worked                    1350 non-null   float64
 9   Employee_Role                           1350 non-null   float64
 10  Annual_Bonus                            1350 non-null   float64
 11  Training_Hours                          1350 non-null   float64
 12  Department                              1350 non-null   float64
 13  Annual_Bonus_Squared                    1350 non-null   float64
 14  Annual_Bonus_Training_Hours_Interaction 1350 non-null   float64
 15  Employee_Turnover                       1350 non-null   int64
dtypes: float64(15), int64(1)
memory usage: 168.9 KB
```

In [8]:
```
df.isna().sum()
```

Out[8]:
```
Job_Satisfaction                          0
Performance_Rating                        0
Years_At_Company                          0
Work_Life_Balance                         0
Distance_From_Home                        0
Monthly_Income                            0
Education_Level                           0
Age                                       0
Num_Companies_Worked                      0
Employee_Role                             0
Annual_Bonus                              0
Training_Hours                            0
Department                                0
Annual_Bonus_Squared                      0
Annual_Bonus_Training_Hours_Interaction   0
Employee_Turnover                         0
dtype: int64
```

In [18]:
```
x=df.drop('Employee_Turnover',axis=1)
y=df['Employee_Turnover']
```

In [21]:
```
from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=42

x_train.shape,x_test.shape,y_train.shape,y_test.shape
```

Out[21]:
```
((1080, 15), (270, 15), (1080,), (270,))
```

In [24]:
```
# Baseline Logistic Regression
from sklearn.linear_model import LogisticRegression
lr=LogisticRegression(max_iter=200)
lr
```

Out[24]:
```
LogisticRegression
LogisticRegression(max_iter=200)
```

In [25]:
```
lr.fit(x_train,y_train)
```

Out[25]:
```
LogisticRegression
LogisticRegression(max_iter=200)
```

In [27]:
```
# L1 Regularization (Lasso)
lasso = LogisticRegression(penalty='l1', solver='liblinear', C=0.5)
lasso.fit(x_train, y_train)
```

Out[27]:
```
LogisticRegression
LogisticRegression(C=0.5, penalty='l1', solver='liblinear')
```

In [29]:
```
# L2 Regularization (Ridge)
ridge = LogisticRegression(penalty='l2', C=1, max_iter=200)
ridge.fit(x_train, y_train)
```

Out[29]:

```
LogisticRegression
LogisticRegression(C=1, max_iter=200)
```

In [31]:

```python
# Model Evaluation
from sklearn.metrics import accuracy_score, classification_report

models = {'Baseline': lr, 'Lasso': lasso, 'Ridge': ridge}

for name, model in models.items():
    y_pred = model.predict(x_test)
    print(f"\n{name}")
    print("Accuracy:", accuracy_score(y_test, y_pred))
    print(classification_report(y_test, y_pred))
```

```
Baseline
Accuracy: 0.8592592592592593
              precision    recall  f1-score   support

           0       0.85      0.90      0.87       145
           1       0.87      0.82      0.84       125

    accuracy                           0.86       270
   macro avg       0.86      0.86      0.86       270
weighted avg       0.86      0.86      0.86       270


Lasso
Accuracy: 0.8703703703703703
              precision    recall  f1-score   support

           0       0.86      0.90      0.88       145
           1       0.88      0.83      0.86       125

    accuracy                           0.87       270
   macro avg       0.87      0.87      0.87       270
weighted avg       0.87      0.87      0.87       270


Ridge
Accuracy: 0.8592592592592593
              precision    recall  f1-score   support

           0       0.85      0.90      0.87       145
           1       0.87      0.82      0.84       125

    accuracy                           0.86       270
   macro avg       0.86      0.86      0.86       270
weighted avg       0.86      0.86      0.86       270
```

In [ ]:

In [ ]: